

# EE284A Lab 4: Powering IoT Systems

May 7, 2026

## 1 Introduction

IoT devices are deployed across many environments and serve a wide range of applications. In this lab series, we have learned to interface with sensors and actuators and explored methods for short- and long-range communication. This is only a fraction of the system-level challenges involved in building real-world IoT devices. In this final lab, we focus on one of the main constraints encountered by IoT devices: **power**. IoT devices are often deployed in resource-constrained or hard-to-access locations, making reliable, long-lasting power a major challengeone that directly affects installation, maintenance, and overall system lifetime. Available energy constrains choices such as communication frequency, transmit power, and which sensors or peripherals are practical. Designers must often choose between battery-powered operation and energy harvesting. Balancing performance and lifetime requires careful accounting of the device's power budget. Limited power can make actuation particularly challenging. We would like our IoT devices to interact with the physical world by performing actions such as turning on a light, taking photographs, driving a motor to unlock a door, or producing an alert sound when movement is detected. Adding actuation increases the usefulness of an IoT system, but it also introduces new constraintspecially because actuators often draw far more current than sensors or computation. This lab explores how power availability, energy harvesting, and actuation must be jointly considered when designing sustainable IoT systems. In this lab, you will use a solar panel to harvest energy and charge a LiPo battery, which will power your microcontroller and a fan. You will need to use energy-aware control strategies to ensure that your system operates on harvested power while maximizing the time the fan is on.

## 2 Learning Objectives

After completing this lab, you will be able to:

1. Measure voltage and current to track power harvested and consumed by the system.
2. Implement solar energy harvesting for an IoT device and evaluate its impact on system capabilities.
3. Use a power budget to estimate and extend system lifetime using power-aware actuation controls.

## 3 Required Parts

- ESP32 Feather V2 Microcontroller
- 1000 mAh LiPo battery
- Breadboard
- [Battery Charging Board](#)
- Adafruit INA219 Current/Voltage monitor

- Two JST breakout cables
- Mini screw driver
- 3.3 V DC fan
- Solar panel
- Servo Motor

## 4 Power Monitoring and Battery Setup

In this section, you will set up a battery charging board that allows us to measure the harvested and consumed power separately. We will use the INA219 to measure the battery voltage and instantaneous current, enabling detection of charging and discharging. The simple expression  $P = IV$  relates the power to the current and voltage readings. Note that batteries exhibit different behavior depending on their charge state. To take measurements during this lab, keep your battery between 3.5 and 3.7 V. Also note that it can be damaged if discharged too deeply, so always keep the battery voltage above 3.2 V. It can be quickly recharged by plugging the battery charging board or the microcontroller into a USB port or discharged by unplugging it from power.

We will also establish a Bluetooth link between our microcontroller and the computer, enabling us to monitor battery status without being plugged in or charging via the computer.

### Experiment

1. Place your microcontroller, battery charging board, and current/voltage monitor (INA219) on our breadboard.
2. Use a JST cable to connect the battery charging board's "Load Out" port to the microcontroller.
3. Cut the JST cable connected to the LiPo battery. You should now have two separate pieces: the LiPo battery with exposed positive and negative terminal wires, and the breakout JST connector, as shown in [Figure 1](#).
4. Plug the breakout JST connector into the "LiPo Batt" port on the battery charging board. Then, connect the positive terminal of the JST breakout cable on the charging board to the negative terminal (Vin-) of the voltage/current monitor.
5. Connect the positive terminal (Vin+) of the voltage/current monitor to the positive terminal of your battery.
6. Connect the negative terminal of the JST breakout cable on the battery charging board, and the battery to a common ground with the microcontroller. After this step, your bread board will look like [Figure 2](#).
7. Refer to the wiring directions at <https://learn.adafruit.com/adafruit-ina219-current-sensor-breakout/wiring> to connect the INA219 board to the microcontroller. Then, modify the example code at <https://learn.adafruit.com/adafruit-ina219-current-sensor-breakout/arduino-code> to read and log the battery voltage and current from the INA219 to the serial monitor every 20 ms. Take a screenshot of the serial monitor with the voltages and currents.
8. Modify the code `bluetooth_tx_starter.ino` (which is on Canvas under Assignments – Lab 4) to send your voltage and current readings over Bluetooth (change the Bluetooth device name so that you can recognize it on your computer here:

```
BLEDevice::init("<Your Bluetooth Device Name>");
```

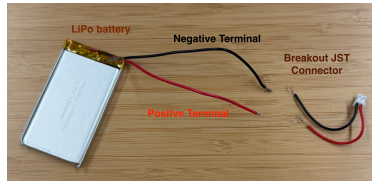


Figure 1: Step3

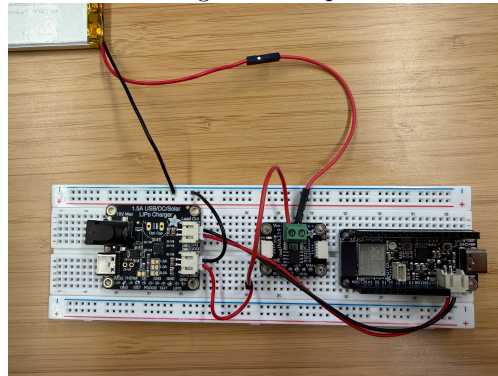


Figure 2: After Step6

9. Run the Python script below on your laptop to find the device address of your Bluetooth device based on the name you gave it using this script. It will output all Bluetooth devices in the environment. Look for the name you gave to your device and it will look something like this: <Your Bluetooth Device Name> 844CE2E8-B9FB-14D1-9FA0-D9F6F5FA946E.

```
import asyncio
from bleak import BleakScanner
async def main():
    print("Scanning...")
    devices = await BleakScanner.discover(timeout=5)
    for d in devices:
        print(d.name, d.address)
asyncio.run(main())
```

10. Modify and run the code `bluetooth_rx_starter.py` (which is on Canvas under Assignments – Lab 4) on your laptop to:
  - Connect to your feather over Bluetooth
  - Read the voltage and current every 20 ms for one minute and save this as CSV file on your computer.
11. Once your firmware has been written, load it on the feather. Run your Python script to start recording your battery voltage and current for one minute and the CSV file with the readings when the microcontroller is plugged and unplugged from the computer.

## Deliverables

1. Include a wiring diagram showing the correct placement of the INA219 to measure the voltage of the battery and the flow of current to the battery.
2. Provide the Arduino code used to get the current and voltage using the INA219 and a screenshot of the serial output. What are the shunt and bus voltages, and how are they used to calculate the voltage at the battery terminal on the battery board?

3. Provide the modified Python code and a screenshot of the voltage and current sent over Bluetooth. Record the battery current and voltage every 20 ms for 1 minute via Bluetooth, with the MCU either plugged into the computer or unplugged. Report the average current and voltage in each case. Explain the voltages and currents you measure in each scenario. What is the instantaneous power in each case?
4. Create a plot of the power every 20 ms as the board was unplugged from the computer. Also plot the same data but averaged for 10 s (you should have six points total). Why does the power vary so much in the 20 ms case? Despite this, when analyzing power consumption, why does it make sense for us to use an average?

## 5 Measuring Device Power Consumption

Now that we can monitor the power flowing in and out of our battery, let's investigate how much power our system uses. Now that Bluetooth is working, **unplug the microcontroller from your laptop** and power it solely from the LiPo battery. All sensor readings will be transmitted to your laptop over Bluetooth from this point on. (Remember to reconnect the USB cable whenever you need to flash new code.)

### Experiment

1. Connect the fan to the 3 V supply from the microcontroller and ground. Record the power average of the fan over one minute, and use your previous measurement of the MCU's power consumption to report only the fan's power consumption.
2. Instead of having the fan on all the time, we would like to be able to control it, so that we can control the discharge rate of our system. Using the simple MOSFET circuit shown below, connect the fan so that it can be turned on and off.

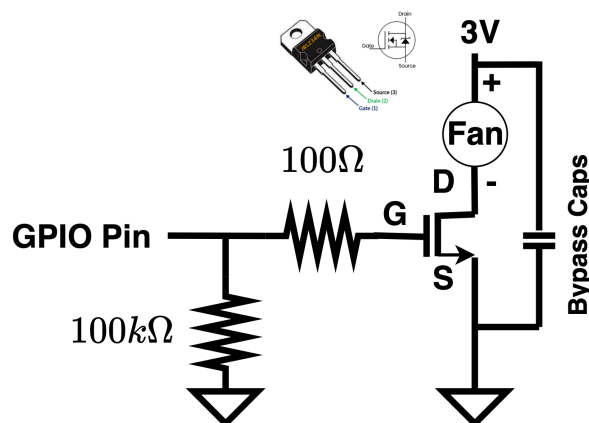


Figure 3: MOSFET Circuit for Fan Control

3. Write firmware to change the duty cycle of the fan to 25% and 50%. Record the average power over one minute for each.

### Deliverables

5. Create a table of the system's average power consumption with: no fan, 100% duty cycled fan, 50% duty cycled fan, and 25% duty cycled fan.
6. Why did we use a MOSFET to control the fan?

7. While this fan is very low-power, why do actuators, such as motors, typically dominate the power budget in IoT systems?

## 6 Measuring Solar Panel Harvesting

Harvesting power from the environment is an effective way to enable greater flexibility in deployment location, and solar panels are a common choice. We will investigate energy harvesting from a low-cost solar panel and analyze how its capabilities affect the performance of our system.

### Experiment

1. Disconnect the fan from your circuit.
2. Connect the solar panel to the charger input and confirm the INA219 indicates charging (the power consumption should be smaller than the one when the solar panel is disconnected). If you do not observe charging, discharge the battery briefly and try again. You can discharge the battery by running the fan at 100% duty cycle while simultaneously logging data over Bluetooth.
3. For each condition below, log one minute averaged values of battery voltage and battery current, and compute average harvested power
  - Indoor lighting
  - In full sunlight
  - In shadow

### Deliverables

8. Report a table of the measured harvested power in each condition. Again, the harvested power can be determined by comparing the battery power measured with the solar panel disconnected to that measured with it connected.
9. Compare the maximum harvested power to the power consumed by the fan, at 100% duty cycle, and the MCU. Is continuous fan operation feasible under your best conditions?
10. Compare the minimum harvested power to the power consumed by the fan, at 100% duty cycle, and the MCU. Is continuous system operation feasible under these conditions?
11. Our charging board has MPPT built in. What is it, and how does it improve energy harvesting? Explain what other factors impact the amount of power harvested by the panel.

## 7 Power-Aware Control

Now your goal is to design a control algorithm that maximizes the time your fan is on, without allowing your system to lose power (i.e., in the ideal case, the fan is at 100% duty cycle and never turned off). We will simulate full-day operation by having you deploy your node for 5 minutes in the sun and 5 minutes in the dark (mimicking the day and night operation). You will then record your power during this deployment, and you must ensure that your system has an equal or higher battery level (as estimated by the voltage) at the end of your experiment. You must report the average power every 10 s and also the raw 20 ms data. This data must be received over Bluetooth, but you may design the system such that it does not always send packets.

## Experiment

1. Implement a power-aware control policy for the fan that responds to energy availability. Your policy must:
  - Protects the battery by ensuring that its voltage remains above 3.2 V.
  - Maximizes the amount of time the fan is on.
  - Ensures that the harvested power is equal to or greater than the received power.
2. Evaluate your controller by deploying for a total of 10 min, where the first is in direct sunlight for 5 min and the second is in darkness for 5 min.

## Deliverables

12. Describe your control strategy and why it should ensure stable operation. Provide your Arduino code and comment where you applied your control strategy.
13. Include plots of battery voltage/current/power and fan state over the duration of your test.
14. What was the charge state of your battery before and after the test? Did you succeed in having equal or greater power? What percentage of the time was your fan on?
15. Discuss any other ideas you have for energy-aware control that you could implement in the future.

Lastly, don't forget to include a link to your code.